

Automated Extraction of Hedge Fund Quarterly Letter Data: End-to-End Workflow, Methodology, and Reproducibility Guide

Table of Contents

1. What this project is doing.....	3
2. What you need before you start.....	4
You need:.....	4
Your folder structure.....	4
3. How to run this in Google Colab (step-by-step).....	4
Step 0. Open Colab.....	5
Step 1. Create the folder and upload PDFs.....	5
Step 2. Paste the full script.....	5
Step 3. Run all the cells, top to bottom.....	5
Step 4. Download the CSV.....	5
4. Deep dive into every section of the code.....	6
Block A. Installing dependencies.....	6
Block B. Imports & config.....	6
Block C. Loading all PDFs.....	8
Block D. Extracting text from PDFs, including OCR fallback.....	8
Block E. Figuring out which quarter the letter talks about.....	9
Block F. Converting a date like “October 16, 2024” into machine-readable “2024-10-16”.....	10
Block G. Extracting the fund name and the manager name.....	11
1. We define a cleaner to normalize weird phrases.....	11
2. We define clean_firm_candidate(cand).....	12
3. We define looks_like_firm(name, manager_name).....	12
4. Finally, we define guess_fund_and_manager(text).....	13
Block H. Extracting the fund’s quarterly performance and any benchmarks the letter mentions.....	13
Block I. Splitting “clean letter text” vs “hedge clauses”.....	15
Block J. Getting the benchmark index performance from actual market data.....	16
Block K. Computing Alpha.....	17
Block L. The main loop that builds the dataset row by row.....	17
Block M. Turning rows into a real table and exporting.....	20
5. How to interpret the final columns (row-by-row meaning).....	20
6. What can go wrong / how to adapt.....	23
1. The OCR might miss data in heavy tables.....	23
2. A new fund might have a different legal name style.....	23
3. Quarter detection might fail on weird phrasing.....	23
4. Yahoo Finance occasionally rate-limits.....	23
Final mental model.....	24

1. What this project is doing

You have quarterly investor letters from different fund managers (e.g. Alluvial, Cedar Creek / Eriksen Capital, Third Point). Those letters are PDFs.

We want to turn those unstructured PDFs into a clean spreadsheet where every row is:

- One manager
- One quarter
- One benchmark comparison (e.g. S&P 500, Russell 2000)

For each row we want:

- Who wrote it (manager name)
- Which fund it is
- Which quarter/year the letter is talking about
- The date of the letter
- The fund's performance that quarter (in %)
- The benchmark's performance that same quarter (in %)
- "Alpha": how much better or worse the fund did vs the benchmark
- The body text of the letter (for future text analytics)
- The legal disclaimers / hedge clauses separately (for compliance / sentiment work)
- The raw benchmark numbers mentioned in the letter, if any
- Which PDF it came from

So the script:

1. Reads all PDFs in a folder
2. Extracts text from each PDF (including OCR fallback for scanned PDFs)
3. Uses a bunch of pattern matching (regular expressions / "regex") to pull out:
 - quarter/year mentioned
 - performance numbers
 - fund name
 - manager name
 - letter date
 - disclaimers/legal text
4. Uses market data from Yahoo Finance to calculate what S&P 500 and Russell 2000 actually did in that same quarter
5. Calculates alpha = fund % - benchmark %
6. Writes everything to a CSV file you can load into Excel, R, etc.

That's the full pipeline.

2. What you need before you start

You need:

- A Google account (so you can use Google Colab)
- The PDF letters you want to analyze (quarterly investor letters from the funds)
- Internet access (because we pull benchmark index data from Yahoo Finance via [yfinance](#))

Your folder structure

In Colab, we expect to have a folder:

```
/content/HedgeFundLetters
```

This folder should contain only the PDFs you want processed:

```
/content/HedgeFundLetters/  
Alluvial-Capital-Management-Q1-2024.pdf  
Alluvial-Capital-Management-Q3-2024.pdf  
CedarCreek-Q2-2024.pdf  
CedarCreek-Q4-2025.pdf  
Third Point Partners-3.pdf  
Third Point Partners-4.pdf  
...etc...
```

You can upload by:

- In Colab left sidebar → “Files” tab → click “Upload” → choose each PDF, OR
- Mount Google Drive and copy PDFs in, if you prefer. But simplest is: upload manually into [/content/HedgeFundLetters](#).

If that folder doesn’t exist yet, create it in Colab with:

```
import os  
os.makedirs("/content/HedgeFundLetters", exist_ok=True)
```

3. How to run this in Google Colab (step-by-step)

Think of Colab like an interactive notebook. You run code in blocks (“cells”) from top to bottom.

Step 0. Open Colab

1. Go to Google Colab (type “google colab” in browser and open).
2. Click **File** → **New Notebook**.

Step 1. Create the folder and upload PDFs

In the first cell, run:

```
import os
os.makedirs("/content/HedgeFundLetters", exist_ok=True)
```

Then use the Files panel on the left-hand side of Colab:

- Navigate to `/content/HedgeFundLetters`
- Upload all your PDF letters there.

Step 2. Paste the full script

Take the full code you have (the one with all the helper functions and the fixes for fund names, alpha, etc.) and paste it into a new Colab cell.

That code already:

- Installs required Python packages
- Defines all helper functions
- Loops through every PDF
- Builds the final dataframe
- Saves `quarterly_letters_structured.csv`

Step 3. Run all the cells, top to bottom

- The first time you run `!pip install ...`, it will download libraries.
- The script will print a preview of your table at the end.

It will also save the CSV file to:

`/content/quarterly_letters_structured.csv`

-

Step 4. Download the CSV

After the script runs, look in the Files panel in Colab for `quarterly_letters_structured.csv`. Right click → Download.

Now you've got your clean dataset locally.

That's it from a "how to use it" standpoint.

The rest of this document explains what the code is doing internally so that:

- You can explain it to someone else
 - You know where to tweak it if a new manager letter has a new layout
-

4. Deep dive into every section of the code

I'm going to walk through the script block by block in logical order.

Block A. Installing dependencies

```
!pip install pdfplumber pytesseract pillow pdf2image yfinance pandas regex python-dateutil
--quiet
```

What this does:

- **pdfplumber**: reads text out of PDFs that have embedded text (i.e. digital PDFs, not just scans)
- **pytesseract**: OCR library wrapper (reads text from images like scanned PDFs)
- **pillow**: basic image library Tesseract needs
- **pdf2image**: converts PDF pages into images so we can OCR them if needed
- **yfinance**: lets us download index prices from Yahoo Finance for S&P 500, Russell 2000, etc.
- **pandas**: data table library; we'll build our spreadsheet in here
- **regex**: advanced regular expressions (pattern matching, text parsing)
- **python-dateutil**: makes it easy to convert "October 16, 2024" into an actual date object

Colab installs these into its temporary environment.

Block B. Imports & config

```
import os
import re
import datetime
import pdfplumber
```

```
import pytesseract
import pandas as pd
import yfinance as yf
from dateutil import parser as dateparser
from pdf2image import convert_from_path
```

We import the tools we just installed.

Then we define things like:

```
INPUT_DIR = "/content/HedgeFundLetters"
```

```
BENCHMARK_TICKERS = {
    "S&P 500": "^GSPC",
    "Russell 2000": "^RUT",
}
```

Meaning:

- **INPUT_DIR** is where all your PDFs live.
- **BENCHMARK_TICKERS** maps human names (S&P 500, Russell 2000) to Yahoo Finance tickers.
 - **^GSPC** = S&P 500 index ticker on Yahoo Finance
 - **^RUT** = Russell 2000 index ticker

We'll use these to compute benchmark performance for the same quarter the fund is reporting.

Then we define a list of “hedge/disclaimer signals”:

```
HEDGE_SECTION_START_MARKERS = [
    "disclosure",
    "confidential",
    "past performance is not",
    "forward-looking statements",
    "this material is not an offer",
    "investing involves risk",
    ...
]
```

These are common legal boilerplate phrases in hedge fund letters. Our idea is:

- Take everything from the first time we see one of these phrases → treat that as “legal/disclaimer”

- Everything before that → treat as main body text

We want these separated because:

- The body text is what you'll analyze for themes ("AI", "macro", "position sizing")
- The hedge clauses are usually copy/paste legal protection and dilute the signal

Block C. Loading all PDFs

```
def get_all_pdfs(folder: str):  
    pdfs = []  
    for fname in os.listdir(folder):  
        if fname.lower().endswith(".pdf"):  
            pdfs.append(os.path.join(folder, fname))  
    pdfs.sort()  
    return pdfs
```

```
PDF_FILES = get_all_pdfs(INPUT_DIR)
```

Plain English:

- Look in the folder.
- Keep only files that end in `.pdf`.
- Sort them alphabetically (so output order is predictable).
- Store full paths (like `/content/HedgeFundLetters/Third Point Partners-3.pdf`) in `PDF_FILES`.

Later, we loop over `PDF_FILES` and process each letter.

Block D. Extracting text from PDFs, including OCR fallback

PDFs come in two flavors:

1. Text-based PDFs (exported from Word/PDF editor). We can read text directly.
2. Scanned PDFs (like a literal scan/photo). We need OCR.

We handle both.

```
def ocr_pdf_page(pil_image):  
    return pytesseract.image_to_string(pil_image)
```


This uses Tesseract OCR to turn an image into text.

```
def extract_text_from_pdf(pdf_path: str) -> str:
    chunks = []
    with pdfplumber.open(pdf_path) as pdf:
        for i, page in enumerate(pdf.pages):
            txt = page.extract_text() or ""
            # Heuristic: if <50 non-space chars, treat as scanned,
            # OCR instead
            if len(re.sub(r"\s+", "", txt)) < 50:
                imgs = convert_from_path(pdf_path, first_page=i+1,
last_page=i+1)
                if imgs:
                    ocr_txt = ocr_pdf_page(imgs[0])
                    chunks.append(ocr_txt)
                else:
                    chunks.append(txt)
            else:
                chunks.append(txt)
    return "\n".join(chunks)
```

What's happening:

- We open the PDF with `pdfplumber`
- For each page:
 - Try to extract text
 - If the page looks basically empty (less than 50 non-whitespace characters), assume it's a scanned image
 - Convert that page to an image (`pdf2image`)
 - Run OCR on that image (`pytesseract`)
- We collect all page texts and join with newlines

Result: `full_text` is a big string with all the letter's text.

Why this matters:

A lot of hedge fund letters are scanned or have performance tables/screenshots, especially older ones or ones that got forwarded. We don't want to silently lose those pages.

Block E. Figuring out which quarter the letter talks about

We need to know if the letter is about Q1, Q2, Q3, or Q4 of which year. The letters usually say things like:

- “During the Third Quarter, Third Point returned 3.9%...”
- “Q4 2024”
- “In the second quarter of 2025”

We define helpers:

```
QUARTER_WORD_TO_LABEL = {
    "first": "Q1",
    "second": "Q2",
    "third": "Q3",
    "fourth": "Q4",
}
```

Then:

```
def guess_quarter_and_year(text: str):
    # Pattern A: "in the third quarter of 2025"
    # Pattern B: "2025 Third Quarter"
    # Pattern C: "Q3 2025"
    # Pattern D: "During the Second Quarter, ..."
    ...
    return quarter_label, quarter_year
```

In plain English:

- We search the letter text with a few regex patterns.
- We try to match any of those common phrasings.
- We return things like:
 - `quarter_label = "Q3"`
 - `quarter_year = 2024`

If we can't find it cleanly, we return ("", `None`) and that row will have blanks.

Block F. Converting a date like “October 16, 2024” into machine-readable “2024-10-16”

Letters usually have a date at the top:

October 16, 2024

We want a standard `YYYY-MM-DD` string so we can sort/filter by time.

```

MONTH_PATTERN = r"(January|February|...|December) "
DATE_REGEX = re.compile(rf"{MONTH_PATTERN}\s+\d{{1,2}},\s+\d{{4}}")

def guess_letter_date(text: str) -> str:
    m = DATE_REGEX.search(text)
    if not m:
        return ""
    dt = dateparser.parse(m.group(0))
    return dt.strftime("%Y-%m-%d")

```

What this does:

- Look for “Month Day, Year” (e.g. "October 16, 2024")
- Use `dateutil` to parse it into a real date
- Reformat to “2024-10-16”

If there’s no obvious date in that format, we leave it blank.

Block G. Extracting the fund name and the manager name

This is one of the trickiest and honestly most valuable parts.

Why it’s tricky:

- Different letters are signed differently.
- Some managers sign as individuals (“Sincerely, Daniel S. Loeb”), some as firms.
- Sometimes the fund is “Alluvial Capital Management, LLC”
- Sometimes it’s “Cedar Creek Partners LLC”
- Sometimes it’s “Third Point Investors Limited,” but the letter starts with “To the Investors in Third Point Investors Limited...”

We want to consistently identify:

- `manager_name`: e.g. “Dave Waters”, “Tim Eriksen”, “Daniel S. Loeb”
- `fund_name`: e.g. “Alluvial Capital Management, LLC”, “Cedar Creek Partners LLC”, “Third Point Investors Limited”

We do three things in sequence to get those.

1. We define a cleaner to normalize weird phrases

```

def normalize_firm_name(name: str) -> str:
    # "To the Investors in Third Point Investors Limited"
    # -> "Third Point Investors Limited"

```

This strips leading phrases like “To the investors in ...” because that’s not actually the fund name.

2. We define `clean_firm_candidate(cand)`

```
def clean_firm_candidate(cand: str) -> str:
    # Removes titles like "CFA", "CEO"
    # Looks for legal-style names:
    #   - "Alluvial Capital Management, LLC"
    #   - "Cedar Creek Partners LLC"
    #   - "Third Point Investors Limited"
    #   - "Third Point Offshore Fund, Ltd."
    # Returns a cleaned version.
```

This function uses regex to grab “proper-noun looking phrases” that end in words like Capital, Partners, Management, LLC, LP, Limited, Offshore Fund, etc. It also trims junk like “, CFA” or “CEO”.

It also specifically knows how to handle “Third Point ...” style names, because Third Point tends to have multiple related entities (Offshore Fund, Investors Limited, Partners L.P.), and they don’t always put “LLC” right under the signature. The PDF text for Third Point often includes a heading like “To the Investors in Third Point Investors Limited,” and we want to reduce that to “Third Point Investors Limited”.

3. We define `looks_like_firm(name, manager_name)`

```
def looks_like_firm(name: str, manager_name: str = "") -> bool:
    # Returns True if this string looks like a fund or firm
    # Returns False if it looks like the manager’s own name
    # Returns False if it’s an index like NASDAQ, S&P 500
    # Returns False if it's "Fourth Quarter 2024 Investor Letter"
```

Reason:

- We don’t want to accidentally label the fund as “NASDAQ”
- We don’t want to say the fund is “Dave Waters”
- We don’t want “Fourth Quarter 2024 Investor Letter” to become the firm name

This function checks for:

- presence of firm-ish words like “capital”, “partners”, “management”, “fund”, “llc”, “limited”, “point”, “creek”
- and also excludes obvious junk.

4. Finally, we define `guess_fund_and_manager(text)`

This is the orchestrator:

```
def guess_fund_and_manager(text: str):  
    # 1. Look near the signature block:  
    #     - Find lines after "Sincerely," or "Best Regards,"  
    #     - Extract manager_name (e.g. "Daniel S. Loeb")  
    #     - Look in those same lines for a firm-like string  
    #  
    # 2. If we still don't have a firm, scan the first ~120 lines of  
the letter  
    #     for a firm-like candidate.  
    #  
    # 3. If still nothing, fall back to the intro sentence  
    #     ("During the Third Quarter, Third Point returned 3.9% ...")  
    #     and grab that "Third Point" name, if it looks like a firm.  
    #  
    # return fund_name, manager_name
```

Why we do it this way:

- Alluvial and Cedar Creek: the signature block after "Sincerely," or "Best Regards," usually contains both the PM's name and the management company, e.g.
"Dave Waters
Alluvial Capital Management, LLC"
That's very clean.
- Third Point: the signature block tends to only give the person's name ("Daniel S. Loeb, CEO"). The legal entity ("Third Point Investors Limited") appears earlier in the letter in an address line like "To the Investors in Third Point Investors Limited". We're scanning the first ~120 lines to catch that.

Because of these steps, you ended up with:

- "Alluvial Capital Management, LLC"
- "Cedar Creek Partners LLC"
- "Third Point Investors Limited"

...which is exactly what we want.

Block H. Extracting the fund's quarterly performance and any benchmarks the letter mentions

Now we want performance.

In each letter, at the top they almost always say something like:

- “During the Fourth Quarter, Third Point returned 9.1%...”
- “For Q2 2024, Cedar Creek Partners LLC increased 4.8%...”
- “Alluvial Capital Management, LLC declined -2.3% in the quarter...”

We want that number as `fund_return_q_pct`.

Also, sometimes they say:

“S&P 500 returned 4.3%, Russell 2000 -9.5%, etc.”

We try to grab those too as `benchmarks_reported_raw`.

That’s what `extract_fund_return_and_benchmarks(text)` does.

```
def extract_fund_return_and_benchmarks(text: str):  
    fund_return = None  
    benchmarks = {}  
    ...  
    return fund_return, benchmarks
```

How it works:

1. It looks for sentences like:
“During the Fourth Quarter, Third Point returned 9.1% ...”
and extracts that `9.1` into `fund_return`.
We’re doing pattern matching with regex to spot keywords like “returned”, “was up”, “increased by”, “fell”, “declined”, etc.
2. If it doesn’t see that pattern, it tries a different sentence shape:
“The fund returned 4.8% in the third quarter”
or
“... was up 2.2% during Q3 ...”
3. Then it tries to get benchmarks. We maintain a list of known benchmark names:
 - `S&P 500`
 - `Russell 2000`
 - `NASDAQ`
 - `MSCI World`
 - etc.
4. We run regex patterns that try to match things like:
 - “compared to 8.1% for the S&P 500”
 - “The NASDAQ and S&P 500 rose 11.2% and 8.2%”
 - “Russell 2000 ... 9.5%”

If we find:

`benchmarks["S&P 500"] = 8.2`

`benchmarks["Russell 2000"] = 9.5`

5. etc.

Why this matters:

- `fund_return_q_pct` is what the manager claims they did that quarter.
 - `benchmarks_reported_raw` is what the manager says the benchmarks did.
 - Sometimes they don't say it. Then `benchmarks_reported_raw` is `{}`.
 - That's fine because we'll calculate benchmark performance ourselves in a later step anyway.
-

Block I. Splitting “clean letter text” vs “hedge clauses”

Hedge funds put long legal disclaimers at the end of the letter. They look like:

"This material does not constitute an offer to sell... Forward-looking statements...
Past performance is not indicative..." etc.

For text analytics (topic modeling, sentiment, etc.), that legal boilerplate is noise.

We split it with `split_hedge_clauses(text)`:

```
def split_hedge_clauses(text: str):
    lower = text.lower()
    cut_idx = None
    for marker in HEDGE_SECTION_START_MARKERS:
        pos = lower.find(marker.lower())
        if pos != -1:
            if cut_idx is None or pos < cut_idx:
                cut_idx = pos

    if cut_idx is None:
        return text.strip(), ""

    main_text = text[:cut_idx].strip()
    hedge_text = text[cut_idx:].strip()

    return main_text, hedge_text
```

How it works:

- We scan for the first appearance of any disclaimer marker like “confidential,” “this material does not constitute an offer,” etc.
- We cut the letter there.
 - `clean_letter_text` = everything before that cut
 - `hedge_clauses_text` = everything from that cut onward

Interpretation:

- `clean_letter_text` = investment discussion, thesis, portfolio commentary
- `hedge_clauses_text` = compliance / legal

You now have two text fields per letter: one for actual content, one for boilerplate.

Block J. Getting the benchmark index performance from actual market data

We don't just trust what the letter says about the S&P 500. We also calculate it ourselves using real price data.

We wrote:

```
BENCHMARK_TICKERS = {  
    "S&P 500": "^GSPC",  
    "Russell 2000": "^RUT",  
}
```

Then:

```
def get_quarter_date_range(quarter_label: str, year: int):  
    # Q1 -> Jan 1 to Mar 31 of that year  
    # Q2 -> Apr 1 to Jun 30  
    # Q3 -> Jul 1 to Sep 30  
    # Q4 -> Oct 1 to Dec 31  
    # returns start_date, end_date in "YYYY-MM-DD"
```

Then we define `get_index_return_for_quarter`:

```
def get_index_return_for_quarter(ticker: str, quarter_label: str,  
    year: int) -> float:  
    start_date, end_date = get_quarter_date_range(quarter_label,  
    year)
```



```
data = yf.download(ticker, start=start_date, end=end_date, ...)

# We'll look at the first and last price in that quarter.
# Return % = (end - start) / start * 100
return pct
```

Details:

- `yfinance` pulls daily prices for that index between (say) 2024-07-01 and 2024-09-30.
- We grab the first day's close and the last day's close.
- Compute percentage change.

We also had to guard against a pandas quirk where `yfinance` sometimes returns a "Series with ticker label" instead of a plain number. We wrote a helper (`first_scalar`) to always coerce it to a float like `4.38` instead of `Ticker ^GSPC 4.38 dtype: float64`.

This gives us a clean numeric quarterly return for S&P 500 and Russell 2000 that we can trust.

Block K. Computing Alpha

Alpha in this context:

How much better (or worse) the fund did vs a benchmark that quarter.

Formula:

```
alpha_pct_q = fund_return_q_pct - benchmark_return_pct_q
```

So if:

- Fund says: "We returned 10.4% in Q4 2025"
 - S&P 500 actually did 2.68% that same quarter
- Then:
- Alpha vs S&P 500 = 10.4% - 2.68% = 7.72 percentage points

This is exactly what a PM will ask:

"Did we beat the Russell? By how much?"

We're giving that per quarter, per benchmark.

Block L. The main loop that builds the dataset row by row

This is where everything comes together.

```
rows = []

for pdf_path in PDF_FILES:
    full_text = extract_text_from_pdf(pdf_path)

    letter_date_iso = guess_letter_date(full_text)
    quarter_label, quarter_year = guess_quarter_and_year(full_text)

    fund_name, manager_name = guess_fund_and_manager(full_text)

    fund_return_q_pct, benchmarks_reported =
extract_fund_return_and_benchmarks(full_text)
    clean_text, hedge_text = split_hedge_clauses(full_text)

    alpha_info = []
    for bench_name, ticker in BENCHMARK_TICKERS.items():
        bench_ret_calc = get_index_return_for_quarter(ticker,
quarter_label, quarter_year)

        if bench_ret_calc is not None:
            bench_ret_final = bench_ret_calc
        elif bench_name in benchmarks_reported:
            bench_ret_final = benchmarks_reported[bench_name]
        else:
            bench_ret_final = None

        alpha_pp = compute_alpha(fund_return_q_pct, bench_ret_final)

        alpha_info.append({
            "benchmark_name": bench_name,
            "benchmark_return_pct_q": bench_ret_final,
            "alpha_pct_q": alpha_pp,
        })

    if not alpha_info:
        alpha_info = [{
            "benchmark_name": None,
            "benchmark_return_pct_q": None,
            "alpha_pct_q": None,
        }]
```

```

for rec in alpha_info:
    rows.append({
        "manager_name": manager_name,
        "fund_name": fund_name,
        "quarter_label": quarter_label,
        "quarter_year": quarter_year,
        "letter_date": letter_date_iso,
        "fund_return_q_pct": fund_return_q_pct,
        "benchmark_name": rec["benchmark_name"],
        "benchmark_return_pct_q": rec["benchmark_return_pct_q"],
        "alpha_pct_q": rec["alpha_pct_q"],
        "clean_letter_text": clean_text,
        "hedge_clauses_text": hedge_text,
        "benchmarks_reported_raw": benchmarks_reported,
        "source_file": os.path.basename(pdf_path),
    })

```

Let's unpack this slowly.

For each PDF:

1. We pull out the raw text.
2. We figure out:
 - When is the letter dated?
 - Which quarter is being discussed?
 - Who wrote it? (manager_name)
 - Which fund / firm is this? (fund_name)
 - What % return did they report for that quarter? (fund_return_q_pct)
 - What benchmarks did they mention? (benchmarks_reported dict)
 - Split the letter text into "clean_letter_text" and "hedge_clauses_text"

Then for each benchmark we care about (S&P 500 and Russell 2000):

- We compute that index's quarterly return using `yfinance`.
- We compute $\alpha = \text{fund} - \text{benchmark}$.
- We stash that into a list called `alpha_info`.

Then we actually create rows.

Important design choice:

We make **one row per (fund, quarter, benchmark)**.

So if one letter talks about Q3 2024, and we care about both S&P 500 and Russell 2000, we'll produce 2 rows:

- Row with benchmark_name = "S&P 500"
- Row with benchmark_name = "Russell 2000"

This is good because:

- You can compare alpha vs S&P separately from alpha vs Russell.
- You can pivot later in Excel / R.

Finally we append each row to `rows`.

Block M. Turning `rows` into a real table and exporting

At the end:

```
df = pd.DataFrame(rows)
print(df.head(20))
```

```
output_csv = "/content/quarterly_letters_structured.csv"
df.to_csv(output_csv, index=False)
print(f"Saved CSV to {output_csv}")
```

- We create a pandas DataFrame from `rows`. Think of `df` like an Excel sheet in memory.
- We print the first 20 rows so you can sanity check.
- We save to CSV on disk.
- You download that CSV from Colab.

That CSV is your deliverable.

5. How to interpret the final columns (row-by-row meaning)

In the CSV, each row has:

manager_name

Example: "Daniel S. Loeb", "Dave Waters", "Tim Eriksen"

This is whoever signed the letter.

fund_name

Example:

- "Third Point Investors Limited"
- "Alluvial Capital Management, LLC"
- "Cedar Creek Partners LLC"

This is the fund / entity / partnership the letter is about. We cleaned it to strip things like "to the investors in...".

quarter_label

Example: "Q3" or "Q4"

This is which calendar quarter the performance refers to.

quarter_year

Example: 2024

This is the year of that quarter.

So "Q3", 2024 means July 1–Sept 30, 2024.

letter_date

Example: "2024-10-16"

This is when the letter was dated, turned into ISO format.

Nice for sorting, filtering.

fund_return_q_pct

Example: 9.1

This is the fund's reported performance for that quarter, in percent.

So 9.1 means +9.1% for that quarter.

If it's negative, e.g. -3.6, that means -3.6% for the quarter.

benchmark_name

Example: "S&P 500", "Russell 2000"

This tells you which benchmark this row is comparing against.

benchmark_return_pct_q

Example: 2.68

This is what that benchmark actually returned during that same quarter, based on Yahoo Finance ticker data.

If we couldn't compute it (rare), we try to fall back to what the manager said in the letter.

If we still can't, it's blank.

alpha_pct_q

Example: 7.72

This is the difference in percentage points:

$\text{fund_return_q_pct} - \text{benchmark_return_pct_q}$.

Interpretation example:

- Fund return for Q4 2025 = 10.4%
- Russell 2000 return for Q4 2025 = 2.63%
- Alpha vs Russell 2000 = $10.4 - 2.63 = 7.77$
That means the fund outperformed Russell 2000 by 7.77 percentage points that quarter.

This is very easy to explain at investment committee.

clean_letter_text

Everything in the letter **before** we hit legal boilerplate like:

- “This material is not an offer...”
- “Forward-looking statements...”
- “Past performance is not indicative...”

This is what you’ll normally want to analyze (themes, holdings discussed, macro views).

hedge_clauses_text

Everything from the first disclaimer onward.

That includes forward-looking statements, offer language, regulatory language.

We keep it, but in its own column, because:

- Compliance might still care.
- You might want to measure how “lawyered up” a given letter is quarter-to-quarter.
- But you probably don’t want it polluting investment narrative text.

benchmarks_reported_raw

This is a dictionary (looks like `{ 'S&P 500': 2.4, 'MSCI World': -0.1 }`) that contains benchmark numbers exactly as reported in the letter text, not computed from Yahoo.

Sometimes it’s `{ }` because:

- The manager didn’t explicitly write “S&P 500 returned X%”
- They showed it only in a table graphic (which is harder to parse)
- Or the PDF formatting confused extraction

This column is mainly for audit / research:

- “Did the manager quote benchmark performance accurately?”
- “Are they cherry-picking benchmarks?”

source_file

The exact filename of the PDF we pulled from.

This gives traceability.

If someone asks “Where did you get that number?” you can point them back to the source PDF.

6. What can go wrong / how to adapt

1. The OCR might miss data in heavy tables

If a fund delivers performance only in a table (not in a sentence), `fund_return_q_pct` might be missing or `benchmarks_reported_raw` might be `{}`.

You could extend `extract_fund_return_and_benchmarks` to also scan line-by-line for patterns like:

```
S&P 500 ..... 8.2%
Russell 2000 ..... 9.5%
```

and grab those.

2. A new fund might have a different legal name style

If you add a new manager whose firm name ends in something not covered (for example “Capital Partners Offshore SPC”), you may need to tweak the `clean_firm_candidate()` regex so we recognize that as a “firm-like” name.

Rule of thumb: if you start seeing `fund_name` come out weird (“and” or “the NASDAQ”), update `looks_like_firm()` rules.

3. Quarter detection might fail on weird phrasing

Most letters say “During the Third Quarter...” or “Q3 2024”.

But if someone writes “For the period ended September 30, 2024”, we don’t yet parse that.

You could extend `guess_quarter_and_year()` to detect “period ended September 30, 2024” and infer that’s Q3 2024. That’s doable, just more regex.

4. Yahoo Finance occasionally rate-limits

If Colab throws an error from `yfinance`, just re-run the cell.

If a row has missing `benchmark_return_pct_q`, that’s usually a download hiccup or missing quarter/year info.

Final mental model

You've effectively built a small extraction pipeline / analyst assistant that:

1. **Reads PDF hedge fund letters** (including scanned ones)
2. **Figures out the who / what / when**
 - Who wrote it?
 - What fund/entity does it represent?
 - Which quarter/year is being discussed?
 - When was it published?
3. **Pulls critical KPIs**
 - Fund return that quarter (%)
 - Benchmark return that quarter (%)
 - Alpha (% points of outperformance)
4. **Separates narrative vs legal**
 - "Here's what we did and why"
 - "Here is 2 pages of legal disclaimers"
5. **Outputs a flat CSV dataset**
 - One row per (fund x quarter x benchmark)
 - Ready for R / Excel / Tableau / PowerPoint